BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position

(0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.

2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS chooses the next node to explore using only g(n), the cost of the path from the start to that node. It always expands the cheapest node so far, so it spreads out in every direction equally, like a growing circle, and it has no idea where the goal is. A* chooses using f(n) = g(n) + h(n), which adds a heuristic estimate of the remaining cost to the goal. A node that is cheap so far but far from the goal gets a high f value and is delayed, while a node that is close to the goal is explored earlier. So A* is guided toward the goal and expands fewer nodes. In my tests it expanded about 90 to 110 nodes while UCS expanded about 280 to 290, and both found the same shortest paths.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is

Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

A heuristic is admissible if it never overestimates the true cheapest cost to the goal. On a grid where the agent moves one cell Up, Down, Left or Right at a cost of 1 per move, every move changes either x or y by exactly 1. So the agent needs at least |x1 - x2| + |y1 - y2| moves to reach the goal, even if there are no walls. Walls and detours can only make the real path longer, never shorter. Therefore Manhattan distance is always less than or equal to the true cost, and so it is admissible. If the heuristic was not admissible and overestimated the cost, A* could rank the nodes on the truly best path as too expensive and ignore them. It would then return a path that is not the shortest, so the optimality guarantee of A* would be lost, even though it might run faster.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan distance would no longer be admissible. With diagonal moves, one step can change both x and y at the same time. For example, from (0, 0) to (3, 3) the agent needs only 3 diagonal moves if each costs 1, but Manhattan distance says 6. That overestimates the real cost, which breaks admissibility, so A* could return suboptimal paths. A better metric is Chebyshev distance, max(|x1 - x2|, |y1 - y2|), which is exact for 8-way movement when every move costs 1. If diagonal moves cost the square root of 2, then octile distance should be used, because Chebyshev would then underestimate the cost. Euclidean distance would also remain admissible, because the straight line is always the shortest distance, but it is weaker, so A* would expand more nodes than with Chebyshev or octile distance.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Using the distance to only the closest food ignores the cost of collecting the rest of the food, so A* has to discover that cost by searching, and it explores many unnecessary states. The state for this problem should be (agent position, set of food remaining). A stronger heuristic is the cost of a minimum spanning tree (MST) over the agent's position and all the remaining food pellets, using Manhattan distance, or true maze distances, as the edge weights. Any plan that eats all the food must connect all of those points, so the MST cost is a lower bound on the real cost. That means it never overestimates and stays admissible. A simpler version is to take the distance to the nearest food plus the distance between the two food pellets that are furthest apart. A tighter alternative, which stays admissible but takes more time to compute, is to take the maximum of the true maze distances from the agent to each remaining food pellet. Using the MST also gives a good tradeoff between accuracy and computing time. In practice, the agent can also plan a full tour by searching in this larger state space with the MST heuristic, or by choosing the next food using a nearest-neighbour order and then improving it.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

FACULTY OF COMPUTING